Adaptive IB + Olympiad Study Planner

Product spec, scheduling logic, setup guide, and build prompts

Prepared for an IB DP1 student targeting June syllabus completion in Physics HL, Maths AA HL, Chemistry HL, and sustained Olympiad progress.

Document purpose

This package is designed to be directly actionable. It gives you: (1) the product definition, (2) the scheduling rules, (3) the recommended tech stack, (4) a Google Stitch → Figma normalization workflow, and (5) copy-paste prompts for code generation.

Version 1.0 • March 2026

Contents

- 1. Executive summary
- 2. Product scope and non-negotiables
- 3. Scheduling engine logic
- 4. Handling added and removed fixed events
- 5. Data model and entities
- 6. UI pages and interaction rules
- 7. Recommended stack and setup
- 8. Google Stitch + Figma workflow
- 9. Master engineering prompt
- 10. Google Stitch prompt
- 11. Figma normalization checklist
- 12. Implementation roadmap
- 13. References

1. Executive summary

This should not be built as a simple calendar. It should be built as a local-first planning system with a calendar interface. The planning engine is the core. The calendar is only the output layer.

The engine must convert long-range goals into weekly quotas and daily blocks while respecting fixed commitments, fatigue, and recovery. For this use case, the highest-value outcome is not decorative scheduling. It is accurate and adaptive workload allocation.

> **Recommended V1 framing**
> A goal-driven scheduler for IB HL sciences/maths and Olympiad work, with manual control over fixed events and automatic replanning when time is gained or lost.

Primary goals for this student

- Finish the full Physics HL syllabus by 30 June.
- Finish the full Maths AA HL syllabus by 30 June.
- Finish the full Chemistry HL syllabus by 30 June.
- Maintain serious Olympiad preparation up to June, then continue toward a much stronger next-cycle level.
- Preserve rest, gym, piano, and other fixed commitments to reduce burnout risk.
- After June, transition from syllabus coverage into heavy question drilling and past-paper work.

V1 must do four things well

Function	What it means in practice
Store commitments	Keep a clean calendar of fixed or semi-fixed events: school, piano, gym, lessons, tests, family events, recovery blocks.
Track progress	Break each subject into units/topics/subtopics with estimated hours, status, mastery, and review state.
Generate study blocks	Assign exact subject, topic, and duration to suitable free slots based on urgency, difficulty, and energy fit.
Replan intelligently	When events are removed or added, or when sessions are missed, redistribute work without collapsing the whole week.

2. Product scope and non-negotiables

The product should be opinionated. The user can override the calendar, but the planner—not the user—should carry the burden of deciding what to study next.

- Local-first by default. The app should work without requiring a backend for V1.

- Manual add/edit/delete for fixed events.
- Automatic conversion of gained or lost time into updated study plans.
- Strong preference for deterministic scheduling logic over LLM-based scheduling.
- One consistent application shell across all pages: same sidebar, same header, same spacing system, same component library.
- Visible weekly feasibility: the user should always know whether the June targets are still realistic.

Non-negotiable planning rules

- No overfilling every free hour. Leave explicit buffer capacity.
- No heavy scheduling late at night by default.
- No treating Olympiad as generic homework. It needs different block types.
- No one-size-fits-all session length; different block templates should exist.
- No hidden logic. The app should show why a task was scheduled.

3. Scheduling engine logic

Use a deterministic, rule-based scheduler first. This is easier to debug, easier to trust, and easier to tune.

The engine should run in six stages:

1. Compute free slots by subtracting fixed commitments and locked recovery time from the calendar.
2. Classify each free slot by duration, time-of-day, and energy quality.
3. Build a candidate task list from unfinished topics, reviews due, and Olympiad block types.
4. Score candidate tasks using deadline pressure, importance, difficulty, mastery weakness, neglect, and slot fit.
5. Assign the highest-scoring compatible tasks to the best slots, while respecting daily workload caps.
6. Reserve weekly buffer space and rerun scheduling when events change or sessions are missed.

Suggested scoring model

Signal	Effect on task priority
Subject priority	Maths AA HL and Physics HL highest; Chemistry HL close behind; Olympiad high but managed carefully.
Deadline urgency	Topics required for June completion rise as the date approaches.
Remaining workload	Subjects with many unfinished hours need enough weekly allocation to stay feasible.
Low mastery	Weak topics rise, especially before the syllabus-completion deadline.
Review due	Topics requiring consolidation gain priority.

Neglect penalty	Subjects not touched recently receive a priority boost.
Energy fit	Hard tasks are penalized if only a low-energy slot is available.
Fragmentation penalty	A topic needing a 90-minute deep block should not be squeezed into 25 minutes.

Recommended subject weights for this specific case

Subject	Relative weight	Notes
Maths AA HL	1.00	Core academic priority and closely linked to Olympiad growth.
Physics HL	1.00	Core academic priority with heavy problem-solving demand.
Chemistry HL	0.90	High priority, but slightly less central than Maths/Physics.
Olympiad	0.85	High-value, but should be constrained by energy and school workload.
English A SL	0.35	Maintain, not dominate, during the June push.
French B SL	0.35	Maintain, not dominate, during the June push.
Geography HL→SL	0.40	Track commitments, but protect time for the main three subjects.

Block types

Block type	Minutes	Use case
Deep Work 120	110–120	Olympiad proofs, hard Maths AA HL, Physics problem solving.
Standard Focus 90	75–90	Normal topic learning and guided exercises.
Drill 60	50–60	Chemistry calculations, worksheet sets, targeted skill drilling.
Review 45	40–45	Consolidation, corrections, flash review, summary notes.
Recovery 30	20–30	Light admin, lesson prep, flashcards, low-friction study.

4. Handling added and removed fixed events

This requirement is essential. The planner must treat the calendar as dynamic, not static.

When a fixed event is removed

- The gained slot should be reclassified by duration and energy.
- The scheduler should first try to place overdue or high-pressure tasks into that slot.
- If no critical task fits, it should pull forward future high-value work or allocate an explicit buffer/recovery block.
- The app should show that the day or week has become more feasible, not simply leave the slot blank unless the user wants blank space.

When a fixed event is added

- The lost slot should trigger replanning immediately.
- The engine should first use existing free buffers before compressing important study blocks.
- Low-priority or low-pressure work should move first.
- If a critical block is displaced, the app should either split it across suitable future slots or displace lighter work.
- If the new event makes the week infeasible, the app should explicitly show the deficit and suggest the least damaging trade-off.

Replanning order

7. Use already-empty buffer slots.
8. Shorten or move low-priority light tasks.
9. Split medium tasks if their learning quality will remain acceptable.
10. Move lower-pressure tasks to later in the week.
11. Only then compress future days, while still respecting burnout limits.

> **Core rule**
> One disrupted session should never trigger a chain reaction that turns the next three days into an impossible schedule.

5. Data model and entities

A clean V1 can remain small. The important part is not quantity of tables; it is having the right entities and explicit relationships.

Entity	Key fields
Goal	id, title, subject/category, deadline, target completion, priority weight
Subject	id, name, default priority, exam mode, color/icon

Topic	id, subject_id, unit, topic, subtopic, est_hours, difficulty, status, mastery, review_due
FixedEvent	id, title, start, end, recurrence, flexibility, category, notes
StudyBlock	id, date, start, end, subject_id, topic_id, block_type, intensity, generated_reason, status
CompletionLog	id, study_block_id, outcome, actual_minutes, perceived_difficulty, notes
WeeklyPlan	week_start, required_hours_by_subject, assigned_hours_by_subject, slack_minutes, risk_flag
Preferences	workload caps, preferred deep-work windows, recovery rules, default block lengths

Status and mastery suggestions

Field	Suggested values
Topic status	Not started / Learning / First pass done / Reviewed / Strong
Mastery	0–5 scale
Flexibility	Fixed / Movable / Optional
Study block status	Planned / Done / Partial / Missed / Rescheduled

6. UI pages and interaction rules

The UI should feel like one product, not six disconnected mockups. Stitch often drifts in navigation and branding across screens, so the app needs a single reusable shell from the start.

Page	Primary purpose
Dashboard	Show today, this week, on-track status, urgent blocks, completion %, and workload risk.
Calendar	Display fixed events, generated study blocks, buffers, and rescheduled work. Support add/edit/delete.
Subjects	Show syllabus maps for Physics, Maths, Chemistry, and Olympiad, with progress and remaining hours.
Weekly Review	Compare planned versus completed hours, carry-over, and risk to the June target.
Settings/Rules	Let the user define preferred deep-work windows, heavy-day caps, rest rules, and subject weights.
Task/Topic Detail	Explain why a block was assigned, what source material to use, and what counts as 'done'.

UI consistency rules

- One permanent sidebar component and one top header component reused everywhere.
- One spacing scale and type scale. No page-specific improvisation.
- One component family for cards, progress bars, tags, buttons, and modals.
- One token system for colors, border radii, shadows, and states.
- All page layouts should be assembled from the same primitives: shell → page header → filter row → main content region.

7. Recommended stack and setup

For V1, use a local-first web app. This reduces operational complexity and makes it easier to keep the planner fast and private.

Layer	Recommendation	Reason
Framework	Next.js + TypeScript	Strong ecosystem, easy component architecture, good fit for future expansion.
UI	Tailwind CSS + shadcn/ui	Fast development with consistent components.
State	Zustand	Lightweight and sufficient for planner state.
Calendar	FullCalendar	Reliable interaction model for calendar views.
Local storage	IndexedDB via Dexie	Local-first persistence without backend complexity.
Date/time	date-fns	Predictable utilities for slot math and recurrence logic.
Charts	Recharts	Optional progress charts for weekly and subject metrics.
Validation	Zod	Safe typed schemas for settings, tasks, and imported/exported JSON.

Suggested repository structure

Suggested folder structure

```
app/
  dashboard/
  calendar/
  subjects/
  weekly-review/
  settings/
```

```
components/
  shell/
  calendar/
  planner/
  subjects/
  ui/
lib/
  scheduler/
    scoring.ts
    slot-classifier.ts
    replanner.ts
    feasibility.ts
  storage/
  dates/
  constants/
types/
seed/
  syllabus/
  olympiad/
public/
```

Initial setup checklist

12. Create a Next.js TypeScript project.
13. Install Tailwind, shadcn/ui, FullCalendar, Dexie, Zustand, date-fns, Recharts, and Zod.
14. Create a design-token file and shared app shell before building individual pages.
15. Seed the syllabus data for Physics HL, Maths AA HL, Chemistry HL, and Olympiad topic groups.
16. Implement the free-slot calculator and scoring engine before fine UI polish.
17. Build the weekly feasibility calculation and display it on the dashboard early.

8. Google Stitch + Figma workflow

Current official Stitch positioning: it can generate UI from natural-language or image inputs, supports iterative refinement, and can move work into Figma; Google also states it can export front-end code. Figma officially supports variables, modes, components, and shared libraries for design-system consistency.

Recommended workflow

18. Use Stitch to generate initial visual concepts for the app shell and key pages.
19. Export or paste the outputs into Figma.
20. Normalize everything in Figma using tokens, components, and a shared library.
21. Freeze the final page shell and component system before code generation.
22. Only then hand the normalized design plus the engineering prompt to the coding model.

Why Figma is mandatory here

- Stitch is excellent for fast ideation, but it is not the right place to lock a scalable design system.
- Figma variables and modes are the correct way to implement reusable tokens for spacing, color, radius, typography, and themes.

- Publishing shared assets/components prevents the navigation/sidebar from drifting across screens.

> **Critical Stitch constraint**
> Treat Stitch as an ideation accelerator, not the final source of truth. The final source of truth for UI consistency should be a normalized Figma file with one app shell and one component library.

Practical Stitch → Figma normalization sequence

Step	Action
1	Generate 4–6 page concepts in Stitch using the app-shell requirement.
2	Pick the strongest direction and export/paste into Figma.
3	Create a Figma Foundations page: colors, type, spacing, radius, shadows, icon sizing.
4	Convert repeated elements into components: sidebar, top bar, page header, KPI card, calendar event, modal, progress card.
5	Create variables/modes for core tokens and apply them across components.
6	Replace page-by-page improvised nav/sidebar with one shell component instance everywhere.
7	Check alignment, spacing, and typography consistency across every page before development handoff.

9. Master engineering prompt

Copy-paste master prompt for your coding model

```
You are building a local-first web app called “Adaptive IB + Olympiad Study Planner”.

Objective:
Build a high-quality, production-structured V1 that helps a DP1 student finish the full Physics HL,
Maths AA HL, and Chemistry HL syllabi by 30 June, while maintaining serious Olympiad preparation and
protecting recovery time.

Hard product requirements:
1. This is NOT a generic calendar.
2. The core is a deterministic planning engine that decides:
   - what subject to study
   - which exact topic/subtopic
   - when to study it
   - for how long
3. The calendar is the presentation layer for fixed events and generated study blocks.
4. The app must be local-first for V1.
5. The design must use ONE consistent app shell across all pages.
6. Do not improvise branding, sidebar structure, spacing, or nav per page.

Tech stack:
- Next.js + TypeScript
- Tailwind CSS + shadcn/ui
- Zustand
- Dexie (IndexedDB)
- FullCalendar
- date-fns
- Zod

Pages required:
- Dashboard
- Calendar
- Subjects
- Weekly Review
- Settings
- Topic / Study Block detail drawer or modal

Core entities:
- Goal
- Subject
- Topic
- FixedEvent
- StudyBlock
- CompletionLog
- WeeklyPlan
- Preferences

Planning logic requirements:
- Calculate free slots by subtracting fixed events and locked recovery from the calendar.
- Classify slots by duration and energy quality.
- Score tasks using:
  priority weight + deadline urgency + remaining workload pressure + low mastery bonus
  + review due bonus + neglected subject bonus - bad slot fit - fragmentation penalty.
- Generate study blocks that fit both slot quality and task intensity.
- Support block types:
  Deep Work 120, Standard Focus 90, Drill 60, Review 45, Recovery 30.
- Show generated_reason on each study block so the user can understand why it was assigned.

Subject priorities for seed config:
- Maths AA HL: 1.00
- Physics HL: 1.00
```

```
- Chemistry HL: 0.90
- Olympiad: 0.85
- English A SL: 0.35
- French B SL: 0.35
- Geography HL/SL transition work: 0.40

Dynamic calendar requirements:
- The user can add, edit, or delete fixed events.
- If a fixed event is removed, the planner must exploit the gained time.
- If a fixed event is added, the planner must compensate for the lost time.
- Replanning order:
  1. use existing buffer slots
  2. move or shorten low-priority light tasks
  3. split medium tasks where acceptable
  4. move lower-pressure tasks later
  5. only then compress future days within workload caps
- Never let one missed or displaced block destroy the realism of the next three days.

Burnout and realism rules:
- Do not overfill every free hour.
- Leave explicit weekly buffer capacity.
- Support max heavy-study blocks per day.
- Avoid assigning heavy deep-work blocks to late-night low-energy slots.
- Olympiad blocks should prefer high-quality slots.
- Show feasibility warnings if the week becomes unrealistic.

UI and design rules:
- Build one reusable AppShell component with fixed sidebar + top header.
- Build page sections from shared primitives only.
- Use a professional academic/planner aesthetic, not a flashy startup landing page.
- Keep dashboards clean, high-density, and readable.
- Show progress bars, required-hours metrics, and risk flags clearly.
- Use consistent spacing, typography, card radius, and color tokens everywhere.

Data and UX requirements:
- Seed sample syllabus/topic data for Physics HL, Maths AA HL, Chemistry HL, and Olympiad
categories.
- Support status values:
  Not started / Learning / First pass done / Reviewed / Strong
- Support study block status:
  Planned / Done / Partial / Missed / Rescheduled
- Allow export/import of planner data as JSON.
- Weekly Review page must compare planned vs completed hours and carry-over.

Implementation order:
1. Types and schemas
2. Local storage layer
3. Shared app shell and design tokens
4. Seed data
5. Free-slot calculation
6. Scoring engine
7. Study block generator
8. Replanner
9. Calendar UI
10. Dashboard and analytics
11. Subject pages
12. Weekly review
13. Settings
14. JSON import/export

Output requirements:
- Produce a clean, multi-file project structure.
- Explain each major file.
- Do not leave placeholders.
- Build the scheduler as readable TypeScript utilities, not hidden magic.
- Include sample seeded data and a demonstrable initial weekly plan.
```

```
- Keep everything ready for direct local development.
```

10. Google Stitch prompt

Copy-paste prompt for Google Stitch

```
Design a desktop-first web app called “Adaptive IB + Olympiad Study Planner”.

Important constraint:
This app must use ONE consistent navigation/sidebar system and ONE consistent visual brand across
every page. Do not redesign the sidebar, top header, card style, or spacing system from page to
page. Use one reusable app shell.

Product context:
The app helps an IB DP1 student finish the full Physics HL, Maths AA HL, and Chemistry HL syllabi by
the end of June, while also maintaining Olympiad math preparation. It must balance aggressive
academic targets with realistic rest and fixed commitments like school, gym, piano, and lessons.

Visual direction:
- Professional, focused, academic planner aesthetic
- Calm, intelligent, premium but restrained
- Not playful, not childish, not flashy
- Desktop productivity app first
- Strong information hierarchy
- Clear progress indicators and workload visibility
- Comfortable spacing and readable data density
- Palette: deep navy, muted blue, slate, off-white, subtle success/warning states
- Rounded but not overly soft
- Use card-based sections with strong alignment

Create a consistent design system for:
- Sidebar navigation
- Top header / page title region
- KPI cards
- Subject progress cards
- Calendar events and study blocks
- Weekly review cards
- Settings panels
- Modals / drawers
- Buttons, tabs, badges, progress bars, inputs

Generate these pages in one coherent product family:
1. Dashboard
   - today’s generated plan
   - this week’s required hours by subject
   - on-track / off-track indicator
   - urgent items
   - workload risk
2. Calendar
   - fixed events
   - generated study blocks
   - visual distinction between locked events, flexible events, and planner-generated work
   - add/edit/delete interactions
3. Subjects
   - Physics HL, Maths AA HL, Chemistry HL, Olympiad
   - unit/topic cards with completion, mastery, and remaining hours
4. Weekly Review
   - planned vs completed
   - carry-over
   - feasibility risk
```

```
   - subject breakdown
5. Settings
   - preferred deep-work times
   - max heavy sessions per day
   - buffer rules
   - subject weights
6. Topic or Study Block detail drawer
   - exact topic
   - estimated duration
   - generated reason
   - source material
   - mark done / partial / missed

Layout rules:
- Use a fixed left sidebar and stable header bar across all pages.
- Use one consistent grid system.
- Reuse identical card patterns where the information type is similar.
- Keep typography, corner radius, shadows, icon style, and filters consistent.
- Favor clarity over decoration.

Deliverable goal:
Generate a cohesive multi-page UI concept that can be exported to Figma and normalized into a design
system without having to redesign navigation or branding from scratch.
```

11. Figma normalization checklist

Once the Stitch concepts are in Figma, standardize them before any coding handoff.

Category	Normalization actions
Foundations	Define color tokens, text styles, spacing scale, radius scale, elevation, icon sizing.
Shell	Create one Sidebar component and one Header component; replace all page-specific variations.
Components	Turn repeated cards, calendar chips, badges, buttons, tabs, modals, and lists into variants.
Variables	Use variables/modes for colors, spacing, and typography wherever possible.
Grid	Standardize container widths, gutters, and page-header spacing.
States	Create consistent success, warning, danger, selection, hover, and disabled states.
Annotation	Label which pieces are reusable components and which are one-off layouts for developers.
Handoff	Freeze the normalized file as the source of truth before code generation.

12. Implementation roadmap

Phase	What to ship
Phase 1	Types, schemas, seeded syllabus data, preferences, and local storage.
Phase 2	Shared app shell, dashboard skeleton, calendar view, and subject list UI.
Phase 3	Free-slot calculator, slot classifier, scoring engine, and initial study-block generator.
Phase 4	Add/edit/delete fixed events with automatic replanning and weekly feasibility logic.
Phase 5	Completion logging, weekly review, carry-over handling, and risk warnings.
Phase 6	JSON import/export, polish, testing, and example data packs.

Most important implementation principle
Build the scheduler before polishing the interface. A beautiful but weak planner is less useful than a plain but trustworthy one.

13. References

- Google Developers Blog (20 May 2025), “From idea to app: Introducing Stitch, a new way to design UIs.” Key points used here: Stitch accepts prompt and image inputs, supports iterative refinement, can paste to Figma, and exports front-end code.
- Figma Help Center, “Guide to variables in Figma.” Key points used here: variables store reusable values, support design systems and modes/themes, and can be applied across designs and prototypes.
- Figma Design Systems product page. Key points used here: shared libraries, standardized styles/components, variables/modes, and component-to-code mapping support consistency and scale.

End of document